Self-Calibrating Adaptive Sequencing for Exact Retrieval from Heterogeneous Multi-Container DNA Storage: Method and Evaluation on Real Read Streams

Sai Mouli
[Affiliation to be completed by the authors]
[City, Country]
[e-mail]

***Abstract*—DNA archives may be split across many physical containers whose sequencing efficiency and failure risk differ and are unknown when retrieval starts. We study how to read an erasure-coded file back exactly from such containers with few sequencing reads. We present a self-calibrating controller that keeps a discrete Bayesian posterior over each container's efficiency (with a failure atom), converts it into strand-recovery hazards through a hierarchical coverage model, sends the next batch of reads to the container with the largest expected progress per cost, and stops once every stripe holds K verified fragments. We evaluate it by replaying public Illumina read streams (ENA PRJEB65931; two pools from different vendors, six runs each, emulating containers) as empirical recovery-time matrices with common random numbers. Against a static allocation tuned in hindsight on the evaluation data, the controller lowers the expected reads to reach 99% exact recovery by 11.6–13.0% on the first pool and 13.9% on the second (multi-container placements; paired bootstrap CIs exclude zero). On the first pool the gain comes mostly from stopping early; on the second, where one container is nearly dead, it comes from learning to avoid that container. The gain is in the mean: the budget needed for the 99th percentile is not smaller, and on the second pool it is 12% larger. Starting from a generic prior with no calibration reads costs $\leq$0.9% on the first pool and 4–5% on the second. Strongly degraded containers are flagged after one to two reads per stored strand. Negative results: stripe-importance weighting adds nothing; efficiency posteriors are over-confident (an exploratory overdispersed likelihood improves coverage on a held-out pool); the completion forecast is under-confident; and a model-based budget is off by -16 to $+6$% on one pool and up to 5$\times$ on the other. We disclose that our calibrated beliefs used evaluation-split consensus statistics and bound the effect with leak-free priors. All work is computational; nothing was synthesised or sequenced.**

***Index Terms*—DNA data storage, adaptive sequencing, Bayesian calibration, erasure coding, coverage depth, sequential decision making, reproducibility**

I. Introduction

DNA has been demonstrated as an archival medium at kilobyte to hundreds-of-megabyte scale [1]–[5]. Retrieval requires sequencing, which samples molecules at random: some strands are read many times, others never, and reads carry substitution, insertion and deletion errors [6], [7]. Systems therefore protect each strand with an inner code, protect groups of strands with an outer erasure code, and sequence to a fixed depth that is expected to recover enough strands [3], [5]. The number of reads needed for decoding—the *coverage depth*—has recently become an object of study in its own right [8], [9].

We expect practical archives not to be a single homogeneous pool: capacity, random access and risk management argue for many physical containers (vials, capsules, pools) that are written, stored and handled separately. This is our working assumption rather than established practice; random-access systems so far address files within a pool [5], [10]. Recent theory considers erasure-coded data partitioned across containers with uniform sampling [11] and expected recovery time when containers fail [12]. Two operational questions remain open in this setting: (i) containers differ—one has been amplified more often, another has aged—and these differences are not known when retrieval starts; (ii) a fixed sequencing plan must pay for the rare slow case on every retrieval.

Problem. Given an erasure-coded file distributed over M containers of unknown and heterogeneous efficiency, some of which may have failed, sequence in batches and decide after each batch which container to read next and when to stop, so that the file is recovered *exactly* (byte-identical, hash-verified) with high probability while minimising the number (cost) of reads.

Contributions. We are explicit about what is new and what is not; a verified prior-art assessment is given in Section II.

1) A *self-calibrating retrieval controller* that couples a discrete Bayesian posterior over container efficiency (including container failure) with a hierarchical strand-recovery hazard model, allocates reads greedily by expected progress per cost and stops on verified decodability (Section IV). Stopping when decodable is known practice [13], and checksum-verified decoding is standard [4]; the container-selection component has no direct precedent we could find in DNA storage, but has analogues in adaptive sampling and bandits (Section II).
2) An evaluation methodology that turns public sequencing runs into *empirical recovery-time matrices*, so that placement and retrieval policies can be compared on real

reads with common random numbers, including a static baseline whose read fractions are tuned on the evaluation data themselves (Section VI).

3) An empirical study on two real pools from different vendors (one public study) and on controlled simulations answering five research questions (generalisation to unfamiliar pools, calibration from limited observations, detection of efficiency changes, cost reduction, and exact recovery under failures), including negative results (Section VII).
4) Supporting channel results: on two independent nanopore flowcells, a strand-specific error component persists across runs and a hierarchical beta-binomial error model predicts consensus success 2.2× more accurately than an i.i.d. model (Section IV-D).

Everything reported is computational: the experiments reuse public sequencing data and simulation; no DNA was synthesised or sequenced for this work.

II. Background and Related Work

DNA storage systems and channels. Storage systems combine constrained encodings with inner and outer codes [2]–[5], [14], [15]. Channel studies show that coverage is overdispersed (approximately negative binomial or log-normal), that a few percent of strands drop out, and that errors arise mostly in synthesis and sequencing [6], [7], [16]. Information-theoretic models treat the channel as sampling with replacement followed by noise [17]–[19].

Coverage depth. Bar-Lev *et al.* define the coverage-depth problem and show that, for a sampler that cannot be controlled, MDS codes and uniform sampling minimise the expected number of reads, which equals $n(H_n - H_{n-k})$ for an $[n, k]$ MDS code [8]. Follow-ups study random access [9], real-data sampling distributions [20], partitioning over containers [11] and container failures [12], and the reliability of MDS-coded retrieval has been analysed [21]. These works analyse *fixed* sampling laws; we study a *controllable* sampler that learns which container to read.

Adaptive sequencing and sequential decisions. Nanopore "Read Until" and adaptive sampling select molecules in real time [22]–[24]; DNA-storage readouts have been sequenced until sufficient coverage was reached [13]. Choosing which container to read so as to complete K-of-N stripes resembles adaptive coupon collection with a controllable sampler, which has analogues in bandit problems [25], [26].

Placement across failure domains. Erasure-coded storage limits the number of fragments of a stripe per failure domain [27]–[29]; correlated failures complicate such placement [30], and Leong *et al.* studied the optimal allocation of coded storage across unreliable nodes and characterised it in several regimes [31]. Our spreading rule (Section IV-E) is a special case of their analysis and is stated only as a design rule.

Consensus and correlated errors. Strands are reconstructed from noisy reads by trace reconstruction [32]–[34] after clustering [35] or learned models [36]; systematic, context-dependent nanopore errors are well documented [37], [38].

Table I compares the closest works. The gap we address is the combination of (a) unknown, heterogeneous container efficiency that must be learned online, (b) container failures, (c) a controllable, adaptive read allocation with a verified stopping rule, and (d) evaluation on real read streams from more than one pool.

III. System Model and Problem Formulation

Fig. 1 shows the architecture, Fig. 2 the model hierarchy and Table II the notation.

A. Storage

A file (or bundle of files, optionally compressed) is split into S stripes of K data fragments of P=32 bytes. A systematic Cauchy Reed–Solomon code [42] over $\mathrm{GF}(2^8)$ extends each stripe to N fragments; any K suffice (MDS). Each fragment becomes a strand carrying a compact address, an address check byte, the payload and an inner Reed–Solomon code with four parity bytes plus a 16-bit CRC, mapped to DNA by a whitened 2-bit code. Logical metadata (code parameters, per-stripe CRC-32, SHA-256 of the blob and of every file) is kept in a manifest outside the DNA. Placement $x_{ij} \in \{0, 1\}$ assigns fragment i to container j. Our default geometry is K=32, N=48.

B. Sequencing and recovery (hierarchical model)

Reads from container j are a Poisson process in that container's *read clock*. A read originates from stored fragment i with probability

$$\pi_{ij} = y_j \, \frac{x_{ij}\, g_i\, h_{ij}}{\sum_{i'} x_{i'j}\, g_{i'}\, h_{i'j}}, \qquad \begin{aligned} g_i &\sim \mathrm{LogN}(-\tfrac{\sigma_g^2}{2}, \sigma_g^2), \\ h_{ij} &\sim \Gamma(k_j, 1/k_j), \end{aligned} \tag{1}$$

where y_j is the usable yield, g_i a strand effect shared by all containers holding i (synthesis-batch bias) and h_{ij} a container-specific effect. A fragment's cluster in container j becomes decodable once it holds t_{ij} reads, with $\mathbb{P}(t_{ij} \le n) = \varphi_j(n)$, the consensus-success curve. Molecules are absent with probability d_j (dropout) and a container fails entirely with probability f_j. Hence the number of container-j reads at which pair (i, j) becomes recoverable is

$$T_{ij} = \Gamma(t_{ij}, 1)/\pi_{ij} \quad (\infty \text{ if absent or failed}). \tag{2}$$

Given cumulative reads $R = (R_j)$, fragment i is recovered iff $\min_j(T_{ij} - R_j) \le 0$, and the file is decodable iff every stripe has $\ge K$ recovered fragments. Integrating (1) over h_{ij} gives negative-binomial read counts, and over g_i (by Gauss–Hermite quadrature) the decodability of a pair after R_j reads,

$$Q_j(\mu) = (1 - d_j)\, \mathbb{E}_g\Big[\sum_{n \ge 0} \mathrm{NB}(n; k_j, g\mu)\, \varphi_j(n) \Big] \tag{3}$$

evaluated at $\mu = R_j \bar{\pi}_{ij}$ with $\bar{\pi}_{ij} = y_j x_{ij} / \sum_{i'} x_{i'j}$ (synthesis dropout, shared by all copies of a fragment, multiplies Q_j in the code).

TABLE I

POSITIONING AGAINST THE CLOSEST PRIOR WORK. Y: YES; N: NO; P: PARTIAL; I: IMPLICIT (EXPECTED READS UNTIL DECODABILITY ANALYSED, NO OPERATIONAL STOPPING RULE); –: NOT STATED. "CONTAINERS" IN [31] ARE STORAGE NODES, AND IN [26] THEY ARE SAMPLING SOURCES ("EXPERTS").

Work	Multi-container	Container failures	Heterog. containers	Correlated errors	Adaptive read allocation	Stopping rule	Real-data evaluation
Lopez et al. 2019 [13]	N	N	N	–	N	P	Y
Gimpel et al. 2023 [7]	N	N	P	N	N	N	Y
Bar-Lev et al. 2025 [8]	N	N	N	N	N	I	N
Gruica et al. 2024 [9]	N	N	N	N	N	I	N
Cao & Chen 2025 [20]	N	N	N	N	N	I	P
Kas Hanna 2026 [21]	N	N	N	N	N	N	N
Levy et al. 2026 [12]	Y	Y	N	N	N	I	N
Kong et al. 2026 [11]	Y	N	P	N	N	I	N
Leong et al. 2012 [31]	Y	Y	N	N	N	N	N
Weilguny et al. 2023 [24]	N	N	–	N	Y	–	Y
Bubeck et al. 2013 [26]	Y	N	Y	N	Y	N	N
This work	Y	Y[a]	Y	P[c]	Y	Y	Y[b]

[a] Failures injected on real read streams; real failure rates are not measured. [b] ENA PRJEB65931 [39] (same-batch aliquots), Zenodo 17473983 [40] and 14296588 [41]; real-data policy results are replays of real read orders (Sec. VI). [c] Correlated-error model validated, but enters the controller only through calibrated consensus curves. Gimpel et al. [7]: heterogeneity characterised across handling conditions of one pool. Cao & Chen [20]: log-normal channel fitted to their own data. Weilguny et al. [24]: genomic loci, not storage containers.

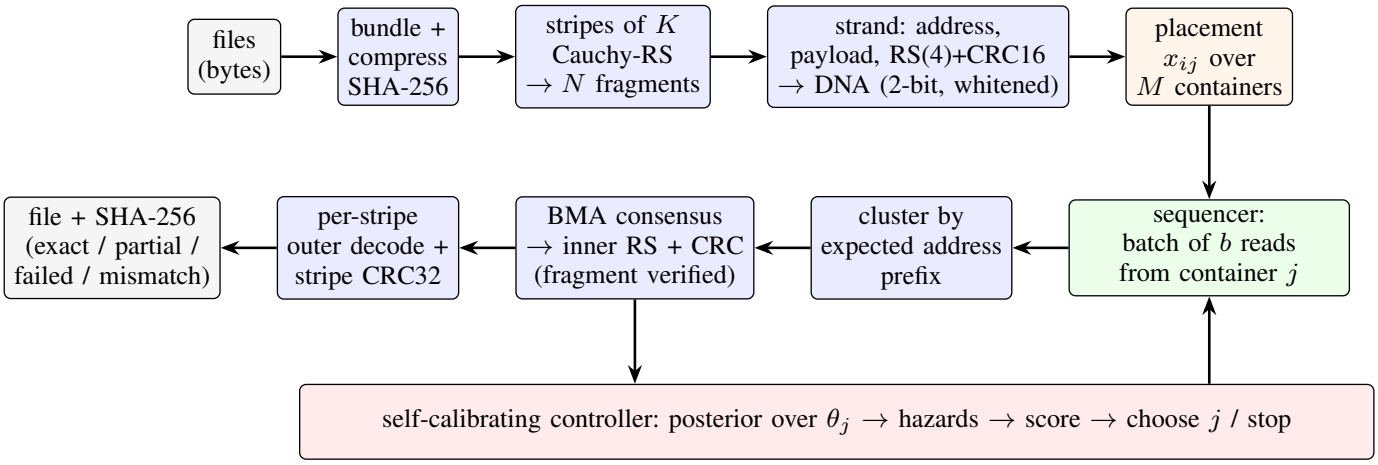


Fig. 1. D3RA-DNA architecture (conceptual diagram). Storage path (top), retrieval path (middle) and the closed control loop (bottom): verified fragment recoveries update the container posterior, which chooses the next container to sequence and decides when to stop.

C. Objective

A retrieval policy π chooses, after each batch, a container and whether to stop. With read costs c_j and a failure budget ε we seek

$$\begin{aligned} &\min_{\pi} \ \mathbb{E}\big[\min(C_\pi, B_\varepsilon)\big] \\ &\text{s.t.} \ \ \mathbb{P}\big(\text{exact recovery within } B_\varepsilon\big) \geq 1-\varepsilon, \end{aligned} \tag{4}$$

where $C_\pi = \sum_j c_j R_j$ at stopping and the cap B_ε is the $(1-\varepsilon)$-quantile of C_π. For a static plan, which commits read fractions w and a budget in advance, the expected spend equals its budget. We report the *expected spend at 0.99* (ε=0.01) throughout. Two caveats apply. Because B_ε is each policy's own quantile, the constraint holds by construction and the objective rewards a low mean even when the tail is heavy; we therefore also report the 0.99-quantile (the budget a user must provision). With $n \leq 100$ trials the empirical 0.99-quantile is essentially the sample maximum.

IV. PROPOSED METHOD

A. Online Bayesian calibration

Each container's unknown deviation from its prior model is summarised by a single efficiency multiplier θ_j that scales its read rate; $\theta_j = 0$ represents a failed container. The support is $\Theta = \{0\} \cup \{\theta^{(1)}, \ldots, \theta^{(25)}\}$ with log-spaced $\theta^{(\ell)} \in [0.01, 5]$ and the prior $p_0(\theta{=}0) = f_j$, $p_0(\theta^{(\ell)}) \propto (1-f_j)\exp(-\frac{1}{2}(\ln\theta^{(\ell)}/s)^2)$, where s (`prior_sd`) encodes how much the container may deviate from the prior model. After R_j reads with U_j verified recoveries among the n_j stored fragments,

$$p(\theta \mid R_j, U_j) \propto p_0(\theta)\,\mathrm{Bin}\big(U_j;\, n_j,\, Q_j(\theta R_j \bar{\pi}_j)\big), \tag{5}$$

with $p(\theta{=}0) = 0$ as soon as $U_j > 0$. This single object absorbs mis-estimated yield, abundance, error rate and container failure. We keep the support down to θ=0.01: with a floor of 0.2, a heavily degraded container whose probe returned

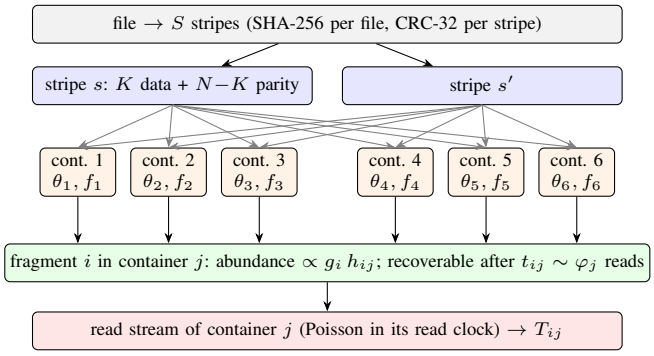


Fig. 2. Hierarchy of the model (conceptual diagram): stripes are spread over containers by the placement x_{ij} (grey arrows); each container has an unknown efficiency θ_j and failure probability f_j; a strand's abundance has a shared (g_i) and a container-specific (h_{ij}) component.

TABLE II
NOTATION AND THE CODE OBJECT IMPLEMENTING EACH QUANTITY (PACKAGE `D3RA`; FULL PATHS IN THE SUPPLEMENTARY MATHEMATICAL SPECIFICATION).

Symbol	Meaning	Code
K, N	data / total fragments per stripe (MDS)	`erasure_codes`
S	number of stripes	`surrogate`
M	number of containers	`surrogate`
x_{ij}	fragment i stored in container j	`Layout`
y_j	usable read yield of container j	`ChannelParams`
g_i, σ_g	shared strand abundance effect, its log-sd	`ChannelParams`
h_{ij}, k_j	container-specific abundance, NB shape	`ChannelParams`
d_j, f_j	molecule dropout, container failure prob.	`ChannelParams`, `FailureModel`
$\varphi_j(n)$	P(consensus correct with n reads)	`ChannelParams`
t_{ij}, T_{ij}	reads needed in cluster; container reads until pair recoverable	`sample_T`, `empirical_T`
$Q_j(\mu)$	pair decodability at mean coverage μ	`_QTable`
R_j, U_j	reads taken, verified recoveries	`AdaptiveController`
θ_j, Θ, s	efficiency multiplier, its grid, prior sd	`AdaptiveConfig`
$h_{ij}(\Delta)$	recovery hazard over Δ reads	`AdaptiveController`
ρ_i, ν_s, W_s	horizon recovery prob., stripe need, stripe weight	`AdaptiveController`
b, H	batch size (reads), look-ahead (batches)	`AdaptiveConfig`
c_j, c_j^{setup}	per-read and per-batch cost	`ChannelParams`
ε, B_ε	failure budget, spend cap	`fixed_policy`, `metrics`
μ_c, ρ	context error rate, error ICC	`correlated_errors`

nothing was classified as failed and abandoned in an earlier version of the controller.

B. Recovery hazards and allocation

For an unrecovered pair, the posterior-plug-in probability of recovery within Δ more reads (the hazard) is

$$h_{ij}(\Delta) = \frac{Q_j(\hat\theta_j(R_j{+}\Delta)\bar\pi_{ij}) - Q_j(\hat\theta_j R_j \bar\pi_{ij})}{1 - Q_j(\hat\theta_j R_j \bar\pi_{ij})}, \qquad (6)$$

with $\hat\theta_j$ the posterior mean. Conditioning on "not yet recovered" automatically lowers the presence probability of long-missing molecules. Let $\nu_s = \max(K - U^{(s)}, 0)$ be the remaining need of stripe s and ρ_i the probability that fragment i is recovered within a look-ahead horizon H (6 batches per container). With $\mu_s = \sum_{i\in s}\rho_i$, $\sigma_s^2 = \sum_{i\in s}\rho_i(1-\rho_i) + \frac{1}{4}$ and $z_s = (\mu_s - \nu_s + \frac{1}{2})/\sigma_s$, the stripe weight is $W_s = \phi(z_s)/(\sigma_s\Phi(z_s))$, the normal approximation of $\partial \log \mathbb{P}(\sum_{i\in s} X_i \ge \nu_s)/\partial\rho_i$ (a normal-approximation sensitivity weight). The controller reads the container maximising

$$S_j = \frac{\sum_{(i,j)\ \text{open}} W_{s(i)}\, h_{ij}(b)\,\big(1 - \rho_i + h_{ij}(b)\big)}{c_j b + c_j^{\text{setup}}}. \qquad (7)$$

The factor $(1-\rho_i+h_{ij})$ is a heuristic: the first-order effect of a batch on fragment i is $h_{ij}(1-\rho_i)$, and the added h_{ij}^2 term keeps fragments that are already likely to be recovered from being scored at zero. ρ_i assumes every container receives H further batches simultaneously. The simpler *coverage-greedy* rule drops W_s and ρ_i: $S_j = \sum h_{ij}(b)/(c_j b + c_j^{\text{setup}})$ over fragments of unfinished stripes.

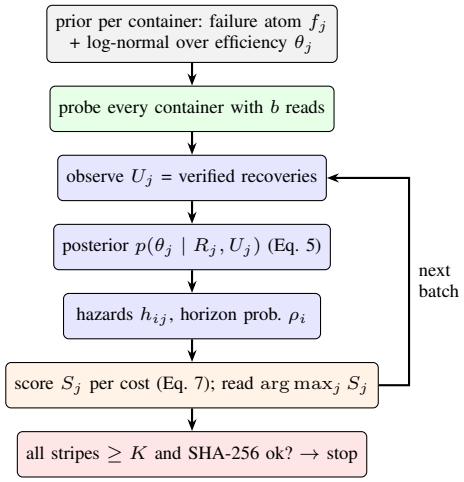


Fig. 3. Self-calibration and allocation loop of the controller (conceptual diagram).

C. Verified stopping and posterior predictive completion

The controller stops with "exact" only when (i) every stripe holds K fragments that each passed inner RS decoding, CRC-16 and the address check, (ii) every stripe matches its per-stripe CRC-32, and (iii) the file matches its SHA-256. If (ii) fails, the controller requests one more fragment for that stripe and localises the faulty fragment by leave-one-out decoding. A wrong file is therefore output only on a SHA-256 collision; the risk of stopping is entirely moved into cost. This verification path runs in the FASTQ pipeline (simulated reads); in the real-data replay (Section VI) a fragment counts as recovered when its consensus equals the designed strand, i.e. decodability is an oracle event. The controller also estimates the posterior predictive probability of completing within the remaining budget,

$$\hat P_{\text{complete}} = \prod_s \Phi\left(\frac{\sum_{i\in s}\rho_i^{\text{rem}} - \nu_s + \frac{1}{2}}{\sqrt{\sum_{i\in s}\rho_i^{\text{rem}}(1-\rho_i^{\text{rem}})}}\right), \qquad (8)$$

with ρ_i^{rem} computed from (6) with the remaining budget spread evenly over all M containers (including any that hold no fragments of a stripe). We evaluate its calibration in Section VII.

D. Correlated errors

Consensus success $\varphi_j(n)$ depends on whether errors repeat across reads of a strand. We model a persistent per-position error propensity

$$p_{sp} \sim \mathrm{Beta}\Big(\mu_{c(s,p)}\tfrac{1-\rho}{\rho},\ (1-\mu_{c(s,p)})\tfrac{1-\rho}{\rho}\Big), \tag{9}$$

where μ_c is the error rate of the local context c (5-mer and homopolymer run) and ρ the intra-class correlation of errors across reads; each read errs at p with probability p_{sp}, so the error count at (s,p) over n reads is beta-binomial. The method-of-moments estimate follows from $\mathbb{E}[(\bar{x}_{sp}-\mu)^2] = \mu(1-\mu)/n + \rho\,\mu(1-\mu)(1-1/n)$. In the experiments the controller uses φ_j calibrated from real reads; the beta-binomial model is evaluated as a predictor of φ (Section VII-A) and is not yet coupled into the controller.

E. Placement rule

For D failure domains failing independently with probability q, an even split of an (N,K) MDS stripe tolerates $t = \lfloor D\rho_{\mathrm{par}} \rfloor$ domain losses, where $\rho_{\mathrm{par}} = (N-K)/N$. If $\rho_{\mathrm{par}} < 1/D$, even spreading is worse than concentrating in one domain for every $q \in (0,1)$; if $t \geq 1$, spreading is better iff $q < q^\star(D,t)$, the unique root of $\mathbb{P}(\mathrm{Bin}(D,q) \geq t{+}1) = q$ (e.g. $q^\star = 0.232$ for $D{=}4, t{=}1$). This follows from the allocation analysis of [31]; a proof, a dropout-margin correction and a heterogeneous extension are given in the supplementary material. A numerical check of the sign prediction is correct in all 8,148 cases with $D \mid N$ (`experiments/reproduce.py`) and in 99.6% of 1,344 cases without that restriction (`theory_summary.json`). We use it only to choose placements.

V. Implementation

D3RA-DNA is a Python package (NumPy, SciPy, `reedsolo`, `edlib`) with 202 unit and integration tests. The full path store → FASTQ → retrieve runs on disk: `d3ra store` writes a manifest, the placement and per-container synthesis FASTA files; `d3ra simulate` writes one FASTQ per container (optionally with real error scripts transplanted from public reads); `d3ra retrieve` streams the FASTQ files in batches (168 nt strands for a 3 kB test archive in the default configuration; the address field grows with archive size), clusters reads against the expected address prefixes (computable from the manifest), runs bitwise-majority-alignment consensus [32], in a two-sided variant of ours (forward-only for raw reads with untrimmed 3′ tails), inner decoding, outer decoding with per-stripe verification, and writes SHA-256-verified files. The controller is vectorised over Monte-Carlo trials. Code, configuration, seeds and result files are listed in Section X.

VI. Evaluation Methodology

A. Empirical recovery-time matrices from real reads

Sequencing reads are, to a first approximation, independent draws from a sample. A container that holds a subset of strands is therefore emulated by the sample's read stream restricted to that subset (plus a proportional share of unmappable reads). The replay is *semi-empirical*: read orders and per-strand read counts are real, consensus success is drawn from per-strand curves estimated from real reads, containers are emulated by subsetting runs of one pool, and only the read order varies between trials. For a real run j and strand i we (1) map reads to designed strands (k-mer voting, edit-distance verification; 99.5–99.7% of reads mapped for Twist, 98.2–99.4% for Genscript), (2) estimate a strand-specific empirical consensus curve $\hat{\varphi}_{ij}(n)$ by repeatedly taking n of the strand's real reads, computing the consensus and checking it against the design, and (3) for each trial draw a random permutation of the run's evaluation reads and set T_{ij} to the stream position of the t_{ij}-th read of strand i, $t_{ij} \sim \hat{\varphi}_{ij}$. The resulting matrix replaces (2); any static or adaptive policy is then replayed on the same matrices (common random numbers). For a static plan with read fractions w the minimal budget of a trial is closed-form, $B^\star(w) = \max_s \mathrm{ord}_K\{\min_j T_{ij}/w_j : i \in s\}$. Beliefs used by the controller take yield, coverage shape and dropout from a disjoint *calibration* split (the first 300k reads of each run; evaluation uses the next 900k). **Disclosure:** the per-container consensus curve φ_j of these "calibrated" beliefs is pooled from the per-strand curves of even-indexed strands, which are estimated from *evaluation*-split reads; the same beliefs choose the single-best container. This is a leak of evaluation information into the controller. Results that are free of it are the generic-φ and generic-prior conditions of Section VII (E4, E10), which we use to bound its effect.

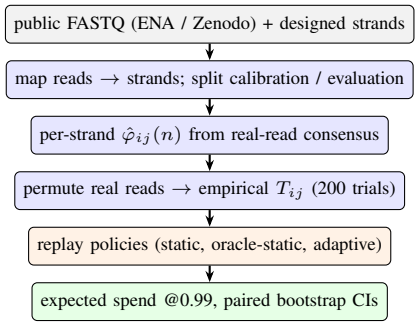


Fig. 4. Experimental pipeline for real-data evaluation (conceptual diagram).

B. Data

Twist pool: six Illumina iSeq runs (ENA PRJEB65931 [7], [39]) of one 12,000-strand (108 nt) pool after aging for 0/2/7 days or 15/60/90 PCR cycles; designs from the DT4DDS release. **Genscript pool:** an independent pool from another vendor (12,472 strands of 102 nt), six runs (aging 0 d replicates A/B, 2 d, 7 d; PCR 15/30 cycles). **Nanopore:** DNAformer binned reads of 55,000 strands on two separate flowcells [36], [40] and the clustered reads of [33], [41]. We place 12,000 strands in 250 stripes of $(K,N) = (32,48)$; each run acts as one container.

TABLE III
EXPERIMENTAL CONFIGURATION (ALL VALUES AS IN THE COMMITTED SCRIPTS).

Item	Setting
Outer code	Cauchy RS over GF(2^8), $(K, N) = (32, 48)$
Inner code	RS, 4 parity bytes + CRC-16 + address check
Real-data layout	250 stripes $\times$ 48 = 12,000 strands
Containers	6 runs per pool, each one container
Read split per run	calibration 300k; evaluation next 900k
Batch b / probe	200 reads / one batch per container
Posterior grid	$\{0\} \cup$ 25 log-spaced points in $[0.01, 5]$
Prior sd s	0.5 (cost runs); 1.0 in change detection; both in E4, E10
Look-ahead H	6 batches per container
Trials (read permutations)	200 (Twist, Genscript round-robin); 100 (Genscript single-best, E4); 60 (failure injection, E10)
Bootstrap	2,000 paired resamples, percentile CIs
Seeds	base seed 0; T matrices seed+7; controller seed+11
Simulation scenarios	s1–s7, 2,000 adaptive / 20,000 static trials (10,000 for static ablation variants) (`experiments/run_all.py`)

C. Baselines and metrics

Fixed depth: uniform read fractions with the budget equal to the 0.99-quantile of its own per-trial requirement on the evaluation data (optimistic: the budget is known in hindsight). *Oracle-tuned static*: read fractions optimised by Nelder–Mead directly on the evaluation matrices (a local optimum; it is not deployable, and static weights combined with verified stopping were not tested). *Adaptive stop, round-robin*: our stopping rule with uniform allocation. *Adaptive (proposed)*: pivotal and coverage-greedy scores. The primary metric is expected spend at 0.99 ((4)); differences are reported with paired bootstrap 95% CIs over the same read permutations (2,000 resamples [43]). Experimental settings are summarised in Table III.

VII. RESULTS

Unless marked *simulated*, every number below is computed from public sequencing reads, for policy costs through the semi-empirical replay of Section VI.

A. Real channels are heterogeneous (RQ2/RQ4 context)

Fig. 5(a) shows real consensus curves of the six Twist containers. Errors are substitution-dominated (0.35–1.15%/nt); three reads give exact consensus for 87–99% of strands depending on handling (Fig. 5a): with one read, 52% of strands give an exact consensus in the fresh sample but 27% after 90 PCR cycles; with three reads, 98% versus 87%. Of the coverage variance beyond sampling noise, 69% is shared by a strand across the six containers (52% in the Genscript pool), an upper bound for aliquots of one synthesis batch. The Genscript pool is harsher: deletion-dominated errors, coverage CV ≈ 1.0 (Twist 0.35), and one container with 54% dropout.

For nanopore reads, Fig. 5(b) evaluates error models fitted on flowcell 1 on held-out strands of flowcell 2. The i.i.d. model over-predicts consensus success (mean absolute error 0.181); the hierarchical beta-binomial model (9) reduces it to 0.083. The strand-specific residual error propensity correlates at r=0.57 between flowcells (null with matched contexts: 0.002); after excluding strands whose design disagrees with the other flowcell's consensus (14%), exact consensus with 10 reads is 0.937 versus 0.997 under i.i.d. errors.

B. Cost of adaptive retrieval (RQ4)

Fig. 6 and Table IV compare policies on the replayed matrices. **Expected spend.** On the Twist pool the adaptive controller needs 6,902–9,114 fewer reads than the oracle-tuned static plan (11.6–13.0% for the multi-container placements, 16.9% for a single container; paired 95% CIs exclude zero for every placement). On the Genscript pool it needs 86.0k fewer reads with round-robin placement (-13.9%, CI -127.1k to -78.8k) and 60.4k fewer with a single container (-21.6%). Because the statistic involves a 0.99-quantile, the bootstrap intervals are asymmetric.

Where the saving comes from. The "adaptive stop, round-robin" column separates stopping from allocation. On Twist (round-robin), stopping alone saves 5.4k reads relative to the oracle-tuned static plan and adaptive allocation a further 2.0k: stopping dominates, and for a single container it is the only mechanism. On Genscript the picture reverses: stopping with round-robin reading is *not* better than the oracle-tuned static plan ($+22.1$k, CI -19.7k to $+30.1$k), because the static oracle already avoids the near-dead container, whereas adaptive allocation saves 108k reads by learning to avoid it. Allocation therefore matters when containers differ strongly. The pivotal score (7) and the coverage-greedy score differ by at most 0.4% with overlapping CIs (no paired test was run); stripe-importance weighting adds nothing measurable.

Tail. The adaptive gain is a gain in the *mean*. The budget that must be provisioned to succeed in 99% of retrievals (q99) is no smaller than the oracle-tuned static budget on Twist (63.2k vs. 63.1k for round-robin) and is 12% *larger* on Genscript round-robin (694.2k vs. 617.9k). A user who pays for reads as they are used benefits; a user who must reserve sequencing capacity for the worst case does not.

Placement. Placing all data in the most efficient container and reading it adaptively needs 44.8k reads on Twist and 219.0k on Genscript, against 67.9k and 810.4k for round-robin placement with fixed depth. This layout offers no protection against losing that container (Section VII-G) and its choice uses the leaky calibrated belief, so we do not present it as a like-for-like saving. A heuristic static allocator that equalises expected coverage is $1.9\times$ *worse* than uniform on Genscript (1,523k vs. 810k) because it over-reads the near-dead container.

C. Self-calibration from limited observations (RQ2)

We replace the controller's belief by progressively less information: a generic literature prior (yield 0.9, NB shape 10, dropout 0.5%, a generic consensus curve; no reads of the pool), beliefs whose coverage parameters are estimated from 1k–300k calibration reads per container (binomial thinning of the calibration split, generic consensus curve), and full

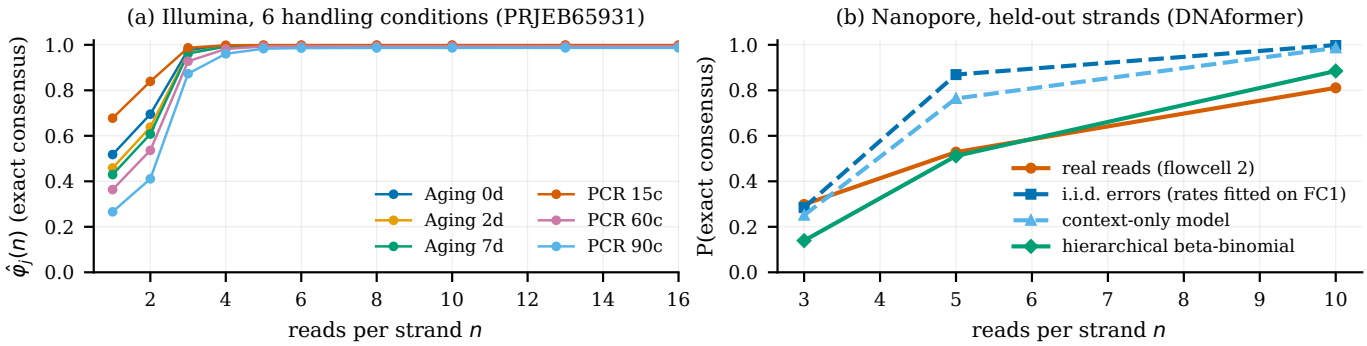


Fig. 5. Real consensus-success curves. (a) Six Illumina containers holding the same Twist pool after different handling. (b) Nanopore: real held-out strands versus error models fitted on the other flowcell.

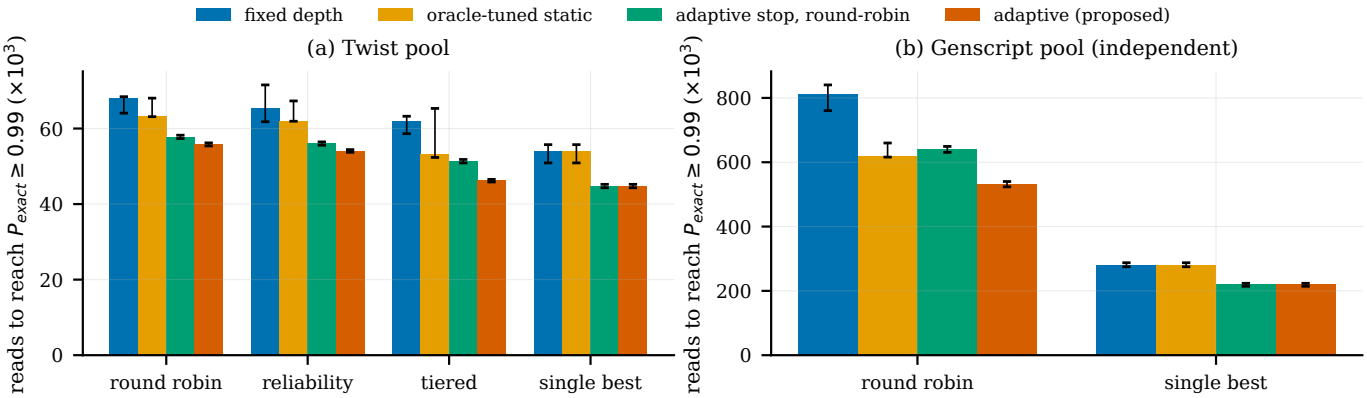


Fig. 6. Reads needed for exact recovery with probability ≥ 0.99 on real read streams (200 read permutations for Twist and Genscript round-robin, 100 for Genscript single-best; error bars: bootstrap 95% CIs).

TABLE IV

EXPECTED READS TO REACH EXACT RECOVERY WITH PROBABILITY ≥ 0.99 ON REPLAYED REAL READ STREAMS (THOUSANDS; BOOTSTRAP 95% CI IN FIG. 6). STATIC PLANS PAY THEIR BUDGET ON EVERY RETRIEVAL, SO THEIR EXPECTED SPEND EQUALS THEIR 0.99-QUANTILE; FOR ADAPTIVE POLICIES THE 0.99-QUANTILE ("Q99", THE BUDGET A USER MUST PROVISION) IS SHOWN SEPARATELY. Δ: ADAPTIVE (PIVOTAL) MINUS ORACLE-TUNED STATIC, PAIRED OVER THE SAME READ PERMUTATIONS (95% CI). WITH A SINGLE CONTAINER, ALLOCATION IS TRIVIAL AND THE GAIN IS DUE TO STOPPING ONLY.

Pool	Placement	Fixed depth	Oracle-tuned static	Adaptive stop, round-robin	Adaptive, cov.-greedy	Adaptive, pivotal	q99 of pivotal	Δ (95% CI)
Twist	round-robin (6)	67.9	63.1	57.8	55.7	55.8	63.2	-7.3 (-12.1, -7.0)
Twist	reliability-weighted	65.4	61.9	56.0	54.2	54.1	61.8	-7.9 (-13.3, -7.6)
Twist	tiered (4)	61.9	53.1	51.3	46.4	46.2	53.0	-6.9 (-19.1, -6.2)
Twist	single best	53.9	53.9	44.8	44.8	44.8	54.0	-9.1 (-11.0, -6.7)
Genscript	round-robin (6)	810.4	617.9	639.9	532.1	531.9	694.2	-86.0 (-127.1, -78.8)
Genscript	single best[†]	279.4	279.4	219.0	219.0	219.0	279.6	-60.4 (-70.5, -54.2)

200 read permutations, except [†]100. All policies completed in every trial. The single-best container is chosen with the calibrated belief, which contains the evaluation-split leak of Section VI. Superseded first-run Genscript numbers (computed before the per-strand consensus cache was regenerated) are kept in `results/realdata_genscript_policies_{roundrobin,part2}.csv`.

calibration (300k reads plus per-run empirical $\hat{\varphi}_j$), each with prior sd $s \in \{0.5, 1.0\}$ (100 read permutations, Twist round-robin). All 14 configurations reach the target in every trial and need 55,254–55,726 reads at 0.99 (Fig. 7); the bootstrap CIs overlap and the largest difference is 0.9%. With zero calibration reads the controller needs 55,584 reads (95% CI 55,134–56,026), against 55,284 (54,800–55,778) with full calibration, and both are well below the oracle-tuned static plan (63,142). Online calibration from the probe and the first batches therefore substitutes for an offline calibration run *on this pool*. We caution that the Twist containers differ only moderately (relative efficiencies 0.34–1.03), so the belief has little room to matter; the harder test is transfer to the Genscript pool (RQ1, below).

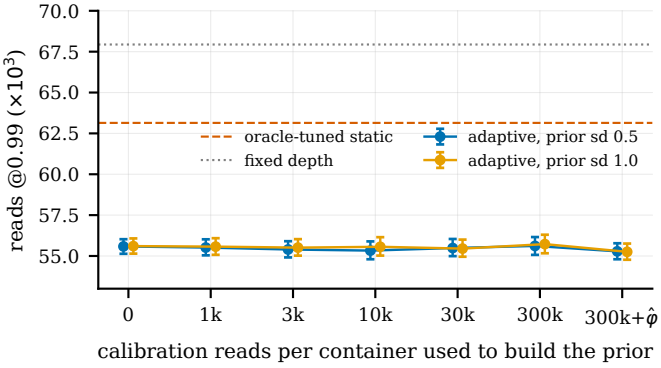


Fig. 7. Self-calibration from limited observations (real Twist streams, 100 permutations): reads at 0.99 versus the calibration reads used to build the controller's prior (0 = generic literature prior). Reference lines: oracle-tuned static and fixed depth (200 permutations).

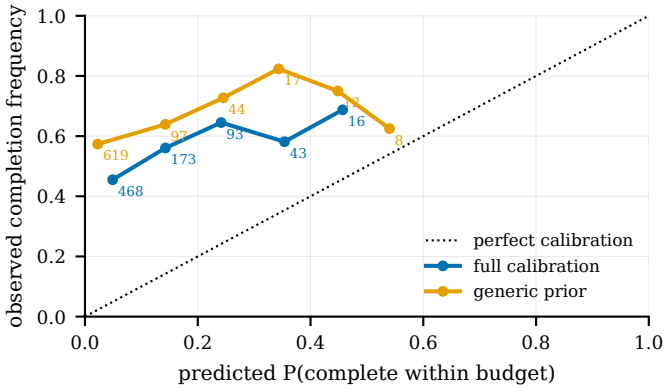


Fig. 8. Reliability diagram of the posterior predictive probability of completing within the budget (real Twist streams, all checkpoints pooled; numbers: trials per bin; bins with fewer than 5 trials omitted).

D. Calibration of the posterior predictive (confidence)

We set the budget to the median adaptive cost on the real Twist streams (55,600 reads), so that about half of the retrievals complete within it, and record $\hat{P}_{\text{complete}}$ (8) after 30, 50, 70 and 85% of the budget (200 trials; beliefs from full calibration or the generic prior). The estimate is strongly *under*-confident (Fig. 8): its mean is 0.001–0.22 while 52% (full calibration) and 60% (generic prior) of retrievals complete; the expected calibration error is 0.30–0.60 and the Brier score (0.31–0.60) is worse than that of a constant forecast (0.24–0.25). It does rank retrievals late in the budget: the AUC for predicting completion rises from chance (0.49–0.56 at 30–50% of the budget, CIs include 0.5) to 0.70–0.71 at 85% (95% CI 0.63–0.77). Thus the posterior predictive is usable for ranking but not as a probability. Its under-confidence is consistent with the behaviour of the futility rule built on it, which abandoned recoverable retrievals in simulation (Section IX), but we did not test that link directly. We have not isolated the cause; candidate explanations (untested) are the spread-evenly allocation assumed for the remaining budget, which is worse than the controller's actual allocation, and the normal approximation in (8).

E. Detecting changes in retrieval efficiency (RQ3)

Table V evaluates the posterior at the read count where the prior model predicts 40% recovery ("$R^\star$", $\approx$ 1–2 reads per stored strand for most containers). In simulation (two containers degraded to relative efficiency 0.14) and on real streams (belief: every container behaves like the fresh pool; the 90-cycle sample has relative efficiency 0.34), the strongly degraded containers are flagged ($\mathbb{P}(\theta < 0.7) > 0.9$) in 100% of trials, and containers within 5% of nominal efficiency are never flagged. Posterior means track the true relative efficiency on the real streams (0.28–0.29 vs. 0.34; 1.06 vs. 1.00–1.03) but *under*-estimate strongly degraded simulated containers (0.05–0.06 vs. 0.14). Moderately reduced containers, which lie above the 0.7 threshold and are therefore *not* degraded by our own definition, are falsely flagged: 2-day aging (0.76) in 10% of trials at $R^\star$, and 60 PCR cycles (0.75) in 73% at $R^\star$ (0% at $2R^\star$). However, nominal 90% credible intervals cover the true efficiency in only 0–83% of trials: the binomial likelihood (5) treats strand recoveries as independent although coverage is correlated through g_i, so the posterior is over-confident.

Exploratory fix (overdispersed likelihood). Replacing the binomial in (5) by a beta-binomial with intra-container correlation ρ_U (a configuration option of the controller) widens the posterior. We chose ρ_U=0.01 after inspecting the Twist and simulated runs (selection set; 200 trials, same seeds, binomial reference re-run in the same process): mean CI coverage rises from 0.08 to 1.00 on the real Twist streams (i.e. the intervals become too wide there) and from 0.51 to 0.79 in simulation, strongly degraded containers are still flagged in 100% of trials, the false alarm on the 60-cycle container disappears (0.82→0; the binomial re-run flags it in 82% of trials versus 73% in Table V, a run-to-run difference we have not fully traced; a candidate is process-dependent seeding, Section X), and mean absolute error of the posterior mean falls (0.060→0.023 real, 0.038→0.027 simulated); ρ_U=0.05 gives 100% coverage everywhere, i.e. intervals that are too wide everywhere. On the *held-out* Genscript pool with ρ_U=0.01 unchanged, mean coverage over five containers rises from 0.01 to 0.85 (range 0.22–1.00), with no false flags before or after; the near-dead PCR_30c container (54% dropout) is flagged in all trials under both likelihoods (its relative efficiency, defined through finite recovery times, is not meaningful and is excluded from coverage). Posterior means on Genscript remain biased upwards (mean absolute error 0.34) because the single multiplier θ_j cannot absorb the mismatch between the fresh-pool belief and other containers' consensus curves. This fix was not used in any cost experiment above, and choosing ρ_U from calibration data remains future work.

F. Generalisation to an unfamiliar pool (RQ1)

We run the controller on the Genscript pool—a different vendor, deletion-dominated errors, coverage CV $\approx$ 1.0 and one near-dead container—with three beliefs: in-pool calibration, a belief transferred from the Twist pool (mean Twist yield,

TABLE V

CHANGE DETECTION AT THE PRIOR-PREDICTED 40%-RECOVERY POINT $R^\star$ (200 TRIALS). FLAG: $\mathbb{P}(\theta_j < 0.7) > 0.9$. COV.: COVERAGE OF THE TRUE VALUE BY THE 90% CREDIBLE INTERVAL.

Setting	Container	reads/strand	true eff.	post. mean	cov.	flag
sim. s7	C0 (degraded)	1.17	0.14	0.06	0.02	1.00
sim. s7	C2 (degraded)	1.66	0.14	0.05	0.00	1.00
sim. s7	C1	1.18	1.02	1.03	0.79	0.00
sim. s7	C5	2.20	0.96	1.02	0.83	0.00
real Twist	Aging 0d	1.08	1.00	1.06	0.05	0.00
real Twist	Aging 2d	1.08	0.76	0.81	0.37	0.10
real Twist	Aging 7d	1.08	0.90	0.82	0.03	0.00
real Twist	PCR 15c	1.08	1.03	1.06	0.00	0.00
real Twist	PCR 60c	1.08	0.75	0.63	0.31	0.73
real Twist	PCR 90c (degraded)	1.08	0.34	0.28	0.00	1.00

TABLE VI

GENERALISATION TO THE UNFAMILIAR GENSCRIPT POOL (60 PERMUTATIONS; READS AT 0.99, THOUSANDS, BOOTSTRAP 95% CI). SHARE: FRACTION OF READS SENT TO THE NEAR-DEAD CONTAINER.

Policy / belief	s	Reads (k)	Share
Fixed depth (uniform, hindsight budget)	–	798.2	16.7%
Oracle-tuned static (hindsight weights)	–	621.0	–
Adaptive, in-pool calibration	0.5	528.9 (513.9–544.4)	3.4%
Adaptive, in-pool calibration	1.0	529.0 (514.0–544.6)	3.4%
Adaptive, transferred from Twist	0.5	557.1 (537.5–578.5)	5.9%
Adaptive, transferred from Twist	1.0	555.8 (535.8–577.4)	5.8%
Adaptive, generic prior (0 reads)	0.5	555.0 (536.3–574.6)	5.5%
Adaptive, generic prior (0 reads)	1.0	551.7 (532.7–571.7)	5.3%

NB shape, dropout and consensus curve, identical for every container), and the generic literature prior (60 read permutations, round-robin placement; static references recomputed on the same 60 matrices; the in-pool and static numbers agree with Table IV, which uses 200 permutations). Table VI shows that every configuration completes in all trials. Without any in-pool calibration the controller needs 551.7–557.1k reads, 4–5% more than with in-pool calibration (528.9k; unpaired bootstrap CIs overlap), yet 10–11% fewer than the oracle-tuned static plan (621.0k) and 30–31% fewer than fixed depth (798.2k). The price of an unfamiliar pool is paid mainly while the posterior learns which container is near-dead: it sends 5.3–5.9% of its reads there, against 3.4% with in-pool calibration. Prior width ($s = 0.5$ vs. 1.0) changes cost by less than 1%. The in-pool belief carries the evaluation-split leak of Section VI; the generic prior does not, so the 4–5% gap is an upper bound on what in-pool calibration (including the leak) buys here. Transfer across vendors thus costs a few percent rather than failing, but on this harsher pool in-pool calibration still saves about 5% (not statistically resolved with 60 trials).

G. Exact recovery under container failures (RQ5)

Removing one container's real read stream (Fig. 9) shows how placement governs degradation. The single-container layout, cheapest without failures (44,473 reads), loses the file when that container fails. A four-container tiered layout (46,530 reads) remains recoverable after any single loss, but losing one of its two main containers leaves few spare fragments per stripe: retrievals then need 127–137k reads and complete within 3× budget in only 45–48% of permutations (losing a third container: 102k reads, always complete). Round-robin over six containers (55,610 reads) degrades by +5 to +32% and always completes. In simulation (12 end-to-end FASTQ retrievals of a 6 kB file per configuration, heavy errors), verification-gated stopping produced no wrong output: with the default inner code (RS with four parity bytes, CRC-16) all 12 retrievals were exact with 0 false fragment accepts in 10,557 decode attempts (95% upper bound 2.8×10^{-4} per attempt); with a deliberately weak code (two parity bytes, no CRC) 45–65 wrong fragments per retrieval (mean 54) passed the inner check, all were caught by the stripe CRC or SHA-256, and the retrievals ended *partial* (11) or *failed* (1), never wrong. Twenty-four retrievals cannot establish a wrong-output rate; the guarantee rests on the SHA-256 check.

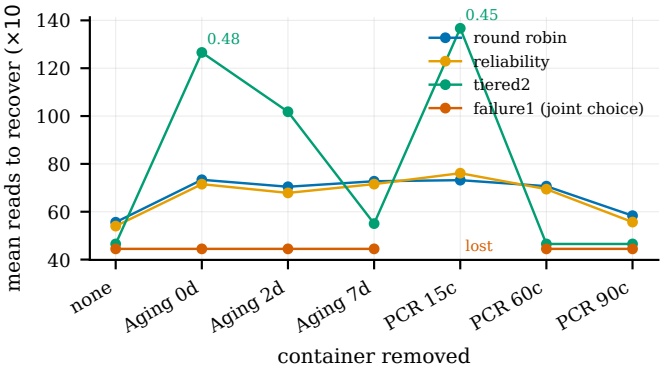


Fig. 9. Real read streams with one container removed: mean reads of successful retrievals; annotations give the completion probability within 3× the 0.95-budget when below 1 ("lost": unrecoverable).

H. Ablation (simulated)

Fig. 10 removes one component at a time in a simulated scenario with correlated failure domains (8 containers in 4 domains; 2,000 trials, 10,000 for the static variants). In mean reads of successful retrievals (the quantity plotted), removing adaptive allocation (round-robin with adaptive stopping) costs +70% (+79% in expected spend at 0.99); static reads cost +67%; uniform or independently optimised placement cost +37–38% with lower reliability; ignoring correlated failures in placement drops the success probability to 0.953. Coverage-greedy scoring is 1.6% cheaper than pivotal scoring. (The "oracle channel" variant is uninformative here because belief and truth coincide in this scenario.) A five-seed sweep showed that the full design is the most reliable design in only 2 of 5 seeds and reaches 0.99 in none of them (0.977–0.989); we do not claim that it meets the target.

VIII. DISCUSSION

What drives the savings. Two mechanisms explain the gain in mean cost: stopping as soon as the data are decodable (a fixed plan must pay for the 0.99-tail on every retrieval) and directing reads away from slow or dead containers once the posterior has seen a few batches. Their weights depend on the pool: stopping dominates when containers are similar (Twist),

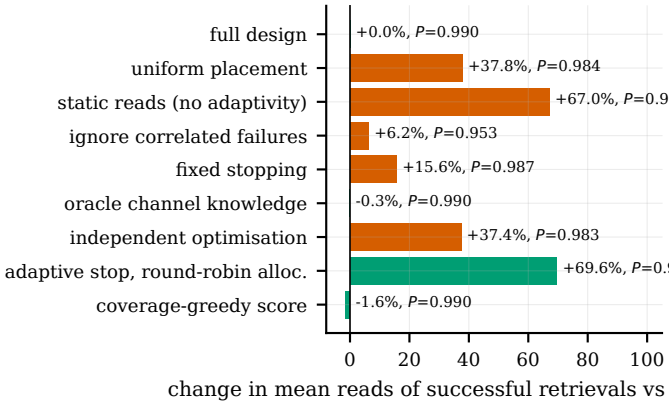


Fig. 10. Ablation in the simulated correlated-failure scenario (s4): change in mean reads of successful retrievals and success probability P (green: $P \geq 0.99$).

allocation when one is nearly dead (Genscript). Neither shortens the tail, so the benefit is to users who pay per read used rather than per reserved run. Stripe-importance weighting adds nothing measurable.

Distribution is not free. When one container is reliable enough for the failure budget, concentrating the data there is cheapest; spreading pays only if container loss must be survived, and the post-failure margin $N-K$ minus the largest per-container load decides how gracefully cost degrades. The choice between these layouts is governed by the failure budget ε, for which real per-container failure statistics do not yet exist.

Model versus reality. Budgets predicted by our analytical surrogate for the static plans are off by -16 to $+6\%$ on the Twist pool; on the Genscript pool the surrogate declares round-robin retrieval infeasible and over-predicts the single-container budget 5.0× (1,397k vs. 279k). Model-based budgets should therefore be calibrated per pool. The controller uses the model only to rank containers and stops on observed decodability, which plausibly makes it less sensitive to such bias; our experiments are consistent with this but do not isolate it.

IX. Limitations and Threats to Validity

Table VII lists assumptions and limitations. The most important are: (i) the six "containers" of each pool are aliquots of one synthesis batch sequenced on two flowcells, not independently synthesised containers, and their coverage is correlated (69% shared variance, an upper bound for same-batch aliquots); (ii) container failures are injected, since no public data report vial-level failure rates; (iii) recovery on real data is measured as exact strand-level consensus against the design, a conservative proxy for inner decoding, because our own encoding has not been synthesised; (iv) read costs are abstract and per-run setup costs are zero, which favours many small adaptive batches (closer to streaming nanopore than to Illumina flowcells); (v) the posterior is over-confident and an early-abandonment rule based on (8) abandoned 3.7–4.5% of retrievals that would have succeeded in a simulated channel-change scenario (s7; not tested on real reads), so it is disabled; (vi) the real-data replay is semi-empirical (Section VI) and treats decodability as an oracle event; (vii) the "calibrated" beliefs contain consensus-curve information from the evaluation split (Section VI); (viii) the real-data cached inputs (per-strand consensus curves) were regenerated between experiment runs, and the policy tables were re-run so that all reported real-data policy numbers use the same inputs (Section X). Internal validity: all policies are compared on identical read permutations; the fixed-depth and static baselines are given their best budgets in hindsight, which favours them. External validity: two Illumina pools (two vendors) from one study and two nanopore datasets.

TABLE VII
Assumptions, limitations and their consequence.

Limitation	Consequence
Containers are aliquots of one synthesis batch	coverage correlated across containers; independent synthesis not tested
Failures injected, not observed	failure budget ε and f_j uncalibrated
Recovery = exact strand consensus vs. design	our encoding not synthesised; inner-code gain not measured on real reads
Abstract read costs, no setup cost	favours small batches; Illumina run economics not modelled
Binomial likelihood ignores overdispersion	over-confident credible intervals (Table V); beta-binomial option fixes most of it on a held-out pool (exploratory, ρ_U chosen post hoc)
Completion forecast $\hat{P}_{\text{complete}}$	under-confident (ECE 0.30–0.60); usable only for ranking late in the budget
Futility (early-abandon) rule	abandoned 3.7–4.5% of recoverable retrievals (simulated s7); disabled
Model bias is pool-dependent	budget error -16 to $+6\%$ (Twist), up to 5× (Genscript)
Adaptive gain is in the mean only	q99 budget not lower (Twist), 12% higher (Genscript RR)
Evaluation leak in calibrated beliefs	bounded by leak-free priors ($\leq$0.9% Twist, 4–5% Genscript)
Semi-empirical replay	decodability is an oracle event; containers emulated by run subsets
Correlated-error model not in controller	enters only through calibrated φ_j
Two Illumina pools from one study	external validity limited
Simulated ablation	component effects not confirmed on real reads

X. Reproducibility and Availability

Code, tests, configuration, fixed random seeds, result files and the scripts that generate every figure and table are in the project repository (Python $\geq$ 3.11; dependencies in `requirements.txt`). Public data are fetched by `experiments/real_data.py download` (ENA PRJEB65931; Zenodo 12666365, 17473983, 14296588). An independent clean-environment reproduction (fresh clone and virtual environment) at an earlier commit, before the RQ1–RQ3 and confidence experiments, passed all its reproduction checks with byte-identical theory outputs (report in the evidence directory). Re-running an experiment reproduces its numbers closely but not bit-for-bit: per-run consensus-curve estimation (and the null permutation of the nanopore study) is seeded through Python's process-randomised string hash, and we observed small run-to-run differences (e.g. a flag rate of 73% vs. 82%) that we have not fully traced. Fixing this requires a stable seed derivation and a full re-run. A claim-to-evidence table mapping every reported number to its code

and result file is provided with the supplementary material (Table VIII summarises it).

XI. Conclusion and Future Work

On replayed real read streams from two pools, a self-calibrating controller that learns container efficiency online, allocates reads by expected progress and stops on decodability lowers the *mean* number of reads needed for exact retrieval by 11.6–13.9% relative to a static plan tuned in hindsight on multi-container placements (17–22% for a single container, where only stopping helps), but does not lower the budget needed for the 99th-percentile retrieval. Stopping provides most of the gain when containers are alike; learning to avoid a near-dead container provides it when they are not. The controller works from a generic prior at a cost of at most 5% and flags strongly degraded containers within one to two reads per stored strand. Its weaknesses are measured: over-confident efficiency posteriors (improved, in an exploratory held-out test, by an overdispersed likelihood), an under-confident completion forecast, no benefit from stripe-importance weighting, a pool-dependent model bias, an evaluation leak in the calibrated beliefs (bounded by leak-free priors) and only semi-empirical, not wet-lab, evidence. Next steps are a wet-lab split-pool experiment with independently synthesised, deliberately degraded and failed containers; estimating the overdispersion from calibration data and recomputing the completion forecast by posterior simulation; tail-aware allocation; coupling the correlated-error model into the controller; and regret guarantees for adaptive container selection.

Acknowledgment

We thank the authors of the public datasets used here (ENA PRJEB65931, DT4DDS; DNAformer; Srinivasavaradhan *et al.*) for releasing reads together with designed sequences.

References

[1] G. M. Church, Y. Gao, and S. Kosuri, "Next-generation digital information storage in DNA," *Science*, vol. 337, no. 6102, p. 1628, 2012.

[2] N. Goldman, P. Bertone, S. Chen, C. Dessimoz, E. M. LeProust, B. Sipos, and E. Birney, "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA," *Nature*, vol. 494, no. 7435, pp. 77–80, 2013.

[3] R. N. Grass, R. Heckel, M. Puddu, D. Paunescu, and W. J. Stark, "Robust chemical preservation of digital information on DNA in silica with error-correcting codes," *Angewandte Chemie International Edition*, vol. 54, no. 8, pp. 2552–2555, 2015.

[4] Y. Erlich and D. Zielinski, "DNA fountain enables a robust and efficient storage architecture," *Science*, vol. 355, no. 6328, pp. 950–954, 2017.

[5] L. Organick, S. D. Ang, Y.-J. Chen, R. Lopez, S. Yekhanin, K. Makarychev, M. Z. Racz, G. Kamath, P. Gopalan, B. Nguyen, C. N. Takahashi, S. Newman, H.-Y. Parker, C. Rashtchian, K. Stewart, G. Gupta, R. Carlson, J. Mulligan, D. Carmean, G. Seelig, L. Ceze, and K. Strauss, "Random access in large-scale DNA data storage," *Nature Biotechnology*, vol. 36, no. 3, pp. 242–248, 2018.

[6] R. Heckel, G. Mikutis, and R. N. Grass, "A characterization of the DNA data storage channel," *Scientific Reports*, vol. 9, no. 1, p. 9663, 2019.

[7] A. L. Gimpel, W. J. Stark, R. Heckel, and R. N. Grass, "A digital twin for DNA data storage based on comprehensive quantification of errors and biases," *Nature Communications*, vol. 14, no. 1, p. 6026, 2023.

[8] D. Bar-Lev, O. Sabary, R. Gabrys, and E. Yaakobi, "Cover your bases: How to minimize the sequencing coverage in DNA storage systems," *IEEE Transactions on Information Theory*, vol. 71, no. 1, pp. 192–218, 2025.

[9] A. Gruica, D. Bar-Lev, A. Ravagnani, and E. Yaakobi, "A combinatorial perspective on random access efficiency for DNA storage," in *Proc. IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2024, pp. 675–680.

[10] J. Bornholt, R. Lopez, D. M. Carmean, L. Ceze, G. Seelig, and K. Strauss, "A DNA-based archival storage system," in *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2016, pp. 637–649.

[11] X. Kong, O. Elishco, C. Wang, and T. M. Duman, "The coverage depth problem in distributed DNA data storage," arXiv:2610.02931 [cs.IT], 2026, preprint.

[12] A. Levy, R. Con, E. Yaakobi, and H. M. Kiah, "Expected recovery time in DNA-based distributed storage systems," arXiv:2602.07601 [cs.IT], 2026, preprint.

[13] R. Lopez, Y.-J. Chen, S. Dumas Ang, S. Yekhanin, K. Makarychev, M. Z. Racz, G. Seelig, K. Strauss, and L. Ceze, "DNA assembly for nanopore data storage readout," *Nature Communications*, vol. 10, no. 1, p. 2933, 2019.

[14] P. L. Antkowiak, J. Lietard, M. Z. Darestani, M. M. Somoza, W. J. Stark, R. Heckel, and R. N. Grass, "Low cost DNA data storage using photolithographic synthesis and advanced information reconstruction and error correction," *Nature Communications*, vol. 11, no. 1, p. 5345, 2020.

[15] W. H. Press, J. A. Hawkins, S. K. Jones, J. M. Schaub, and I. J. Finkelstein, "HEDGES error-correcting code for DNA storage corrects indels and allows sequence constraints," *Proceedings of the National Academy of Sciences*, vol. 117, no. 31, pp. 18 489–18 496, 2020.

[16] Y.-J. Chen, C. N. Takahashi, L. Organick, C. Bee, S. D. Ang, P. Weiss, B. Peck, G. Seelig, L. Ceze, and K. Strauss, "Quantifying molecular bias in DNA data storage," *Nature Communications*, vol. 11, no. 1, p. 3264, 2020.

[17] I. Shomorony and R. Heckel, "DNA-based storage: Models and fundamental limits," *IEEE Transactions on Information Theory*, vol. 67, no. 6, pp. 3675–3689, 2021.

[18] N. Weinberger and N. Merhav, "The DNA storage channel: Capacity and error probability bounds," *IEEE Transactions on Information Theory*, vol. 68, no. 9, pp. 5657–5700, 2022.

[19] A. Lenz, P. H. Siegel, A. Wachter-Zeh, and E. Yaakobi, "Coding over sets for DNA storage," *IEEE Transactions on Information Theory*, vol. 66, no. 4, pp. 2331–2351, 2020.

[20] R. Cao and X. Chen, "Optimizing sequencing coverage depth in DNA storage: Insights from DNA storage data," in *Proc. IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2025.

[21] S. Kas Hanna, "On the reliability of information retrieval from MDS coded data in DNA storage," *IEEE Transactions on Molecular, Biological, and Multi-Scale Communications*, vol. 12, pp. 585–598, 2026.

[22] M. Loose, S. Malla, and M. Stout, "Real-time selective sequencing using nanopore technology," *Nature Methods*, vol. 13, no. 9, pp. 751–754, 2016.

[23] A. Payne, N. Holmes, T. Clarke, R. Munro, B. J. Debebe, and M. Loose, "Readfish enables targeted nanopore sequencing of gigabase-sized genomes," *Nature Biotechnology*, vol. 39, no. 4, pp. 442–450, 2021.

[24] L. Weilguny, N. De Maio, R. Munro, C. Manser, E. Birney, M. Loose, and N. Goldman, "Dynamic, adaptive sampling during nanopore sequencing using Bayesian experimental design," *Nature Biotechnology*, vol. 41, no. 7, pp. 1018–1025, 2023.

[25] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, no. 3-4, pp. 285–294, 1933.

[26] S. Bubeck, D. Ernst, and A. Garivier, "Optimal discovery with probabilistic expert advice: Finite time analysis and macroscopic optimality," *Journal of Machine Learning Research*, vol. 14, no. 17, pp. 601–623, 2013. [Online]. Available: https://www.jmlr.org/papers/v14/bubeck13a.html

[27] S. Weil, S. Brandt, E. Miller, and C. Maltzahn, "CRUSH: Controlled, scalable, decentralized placement of replicated data," in *Proceedings of the 2006 ACM/IEEE Conference on Supercomputing (SC'06)*. IEEE, 2006, p. 31.

[28] A. Cidon, S. Rumble, R. Stutsman, S. Katti, J. Ousterhout, and M. Rosenblum, "Copysets: Reducing the frequency of data loss in cloud storage," in *2013 USENIX Annual Technical Conference (USENIX ATC '13)*. San Jose, CA, USA: USENIX Association, 2013, pp.

TABLE VIII

CLAIM-TO-EVIDENCE SUMMARY (FULL TABLE: `EVIDENCE/EVIDENCE_TRACEABILITY.CSV`). "REAL REANALYSIS" = COMPUTATION ON PUBLIC SEQUENCING READS; NO RESULT IS WET-LAB EVIDENCE PRODUCED FOR THIS WORK.

Claim	Experiment / script	Result file	Status
Adaptive mean cost -11.6 to -16.9% vs. oracle-tuned static (Twist); q99 not lower	`real_data.py policies`	`realdata_policies_v2.csv`	verified, replay of real reads
Adaptive mean cost -13.9% (RR), -21.6% (single) vs. oracle static; q99 +12% (Genscript)	`real_data.py policies` (Genscript)	`realdata_genscript_policies_{rr,sb}_v2.csv`	verified, replay of real reads
Pivotal score no better than coverage-greedy	real-data policies; ablation	`realdata_policies.csv`, `ablations.csv`	verified, negative
Zero-calibration prior matches full calibration (RQ2)	`paper_experiments.py e4`	`paper_e4_limited_calibration.csv`	verified, replay of real reads
Degraded containers flagged; CIs over-confident (RQ3)	`paper_experiments.py ed`	`paper_ed_change_detection.csv`	verified (sim. + real); negative part
$\hat{P}_{\text{complete}}$ under-confident, AUC $\leq$0.71	`paper_experiments.py ec`	`paper_ec_*.csv`	verified, negative
Cross-pool transfer (RQ1)	`paper_experiments.py e10`	`paper_e10_cross_pool.csv`	verified, replay of real reads
Degradation under container loss (RQ5)	`real_data.py failure_injection`	`realdata_failure_injection.csv`	verified; failures injected
No wrong output; 0/10,557 false accepts	`q4_stopping_safety.py`	`q4_summary.json`	verified, simulated channel
Futility rule harmful	`q4_stopping_safety.py`	`q4_futility.csv`	verified, negative
Component ablation	`run_all.py`	`ablations.csv`	simulated
Beta-binomial errors: MAE 0.083 vs. 0.181	`q3_correlated_errors.py`	`q3_summary.json`	verified, real nanopore
Spreading threshold $q^\star$	`theory_checks.py`	`theory_summary.json`	theoretical (known result)
Calibrated beliefs free of evaluation data	`belief_from_calibration`	–	violated (leak disclosed; bounded)
Correlated-error likelihood in controller; regret bounds	–	–	proposed future work
Independent synthesis; observed vial failure rates; currency cost	–	–	unsupported

37–48. [Online]. Available: https://www.usenix.org/conference/atc13/technical-sessions/presentation/cidon

[29] D. Ford, F. Labelle, F. I. Popovici, M. Stokely, V.-A. Truong, L. Barroso, C. Grimes, and S. Quinlan, "Availability in globally distributed storage systems," in *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*. Vancouver, BC, Canada: USENIX Association, 2010. [Online]. Available: https://www.usenix.org/conference/osdi10/availability-globally-distributed-storage-systems

[30] S. Nath, H. Yu, P. B. Gibbons, and S. Seshan, "Subtleties in tolerating correlated failures in wide-area storage systems," in *3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. San Jose, CA, USA: USENIX Association, 2006. [Online]. Available: https://www.usenix.org/conference/nsdi-06/subtleties-tolerating-correlated-failures-wide-area-storage-systems

[31] D. Leong, A. G. Dimakis, and T. Ho, "Distributed storage allocations," *IEEE Transactions on Information Theory*, vol. 58, no. 7, pp. 4733–4752, 2012.

[32] T. Batu, S. Kannan, S. Khanna, and A. McGregor, "Reconstructing strings from random traces," in *Proceedings of the 15th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, New Orleans, LA, USA, 2004, pp. 910–918.

[33] S. R. Srinivasavaradhan, S. Gopi, H. D. Pfister, and S. Yekhanin, "Trellis BMA: Coded trace reconstruction on IDS channels for DNA storage," in *Proc. IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2021, pp. 2453–2458.

[34] O. Sabary, A. Yucovich, G. Shapira, and E. Yaakobi, "Reconstruction algorithms for DNA-storage systems," *Scientific Reports*, vol. 14, no. 1, p. 1951, 2024.

[35] C. Rashtchian, K. Makarychev, M. Racz, S. Ang, D. Jevdjic, S. Yekhanin, L. Ceze, and K. Strauss, "Clustering billions of reads for DNA data storage," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/ab7314887865c4265e896c6e209d1cd6-Abstract.html

[36] D. Bar-Lev, I. Orr, O. Sabary, T. Etzion, and E. Yaakobi, "Scalable and robust DNA-based storage via coding theory and deep learning," *Nature Machine Intelligence*, vol. 7, no. 4, pp. 639–649, 2025.

[37] C. Delahaye and J. Nicolas, "Sequencing DNA with nanopores: Troubles and biases," *PLOS ONE*, vol. 16, no. 10, p. e0257521, 2021.

[38] R. R. Wick, L. M. Judd, and K. E. Holt, "Performance of neural network basecalling tools for Oxford Nanopore sequencing," *Genome Biology*, vol. 20, no. 1, p. 129, 2019.

[39] FML, ETH Zurich, "DT4DDS – characterization of errors and biases in DNA data storage," European Nucleotide Archive, study PRJEB65931, 2023, sequencing data of Gimpel et al., Nat. Commun. 14:6026 (2023); first public 2023-09-08. [Online]. Available: https://www.ebi.ac.uk/ena/browser/view/PRJEB65931

[40] Technion – Israel Institute of Technology, "Binned Nanopore/Illumina reads from DNAformer, split by the data type," Zenodo dataset, 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17473983

[41] D. Bar-Lev, I. Orr, O. Sabary, T. Etzion, and E. Yaakobi, "Prepared binned DNA data storage datasets for reconstruction benchmarking," Zenodo dataset, 2024, re-packaging that includes the clustered nanopore reads released with Srinivasavaradhan et al. (ISIT 2021). [Online]. Available: https://doi.org/10.5281/zenodo.14296588

[42] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.

[43] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Springer US, 1993.